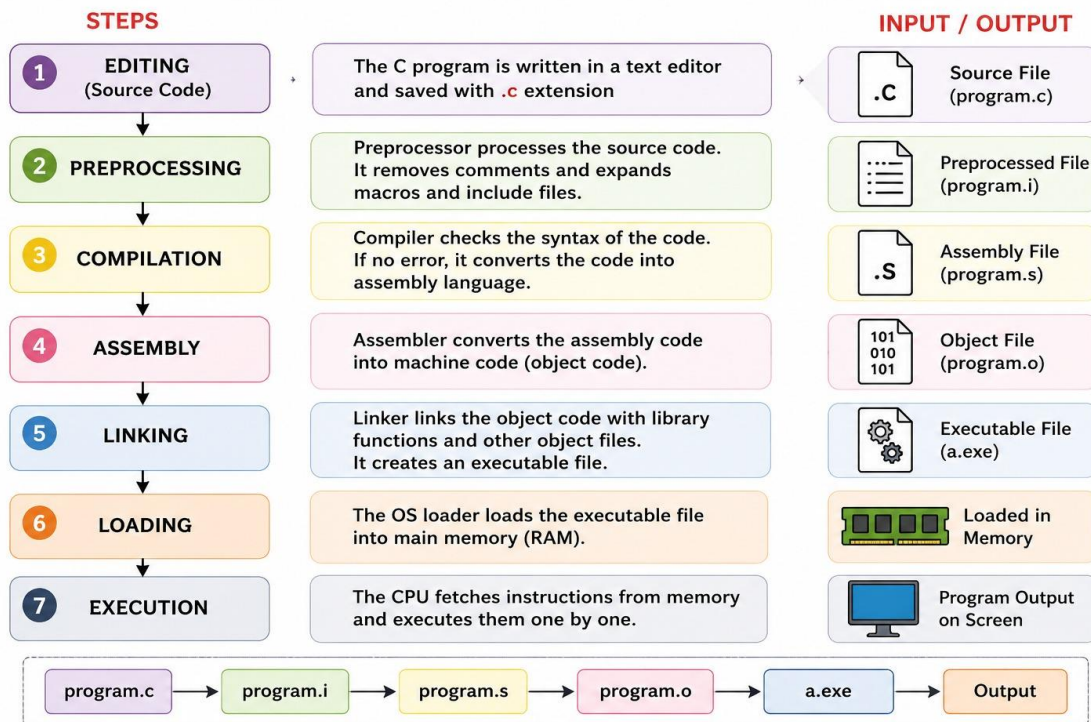


EXECUTION STEPS OF A C PROGRAM



Data Types, Variables, Constants and Operators in C

1. Data Types

A **data type** specifies the type of data that a variable can store.

Basic Data Types in C

Data Type	Meaning	Example
int	Integer numbers	10, -5, 100
float	Decimal numbers	10.5, 3.14
double	Large/precise decimal numbers	12.345678
char	Single character	'A', 'b', '5'

Examples

```
int age = 18;
```

```
float marks = 85.5;
```

```
double price = 1250.5678;
```

```
char grade = 'A';
```

2. Variables

A **variable** is a named memory location used to store a value that can be changed during program execution.

Syntax

```
data_type variable_name;
```

Example

```
int age;
```

```
float marks;
```

```
char grade;
```

Variable Initialization

Assigning a value to a variable at the time of declaration.

```
int age = 18;
```

```
float marks = 85.5;
```

```
char grade = 'A';
```

Changing the Value

```
int age = 18;
```

```
age = 20;
```

The value of age changes from 18 to 20.

Rules for Naming Variables

1. Variable name can contain **letters, digits and underscore (_)**.
2. It cannot start with a digit.
3. Spaces are not allowed.
4. C keywords cannot be used as variable names.

5. C is **case-sensitive**.

Valid

age

student_age

marks1

total_marks

Invalid

1age

student age

float

3. Constants

A **constant** is a value that does not change during program execution.

Types of Constants

Integer Constant

10

25

-50

Floating-Point Constant

3.14

10.5

-2.5

Character Constant

'A'

'b'

'5'

String Constant

"Hello"

"Computer Science"

Using const

The const keyword is used to create a variable whose value cannot be changed.

```
const int MAX = 100;
```

```
const float PI = 3.14;
```

Example:

```
const int x = 10;
```

```
x = 20; // Error
```

4. Operators

An **operator** is a symbol used to perform an operation on one or more values.

Example:

a + b

Here, + is an operator.

5. Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations.

Operator Operation		Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b

Operator	Operation	Example
----------	-----------	---------

%	Remainder (Modulus)	a % b
---	---------------------	-------

Example

```
int a = 10, b = 3;
```

```
printf("%d", a + b); // 13
```

```
printf("%d", a - b); // 7
```

```
printf("%d", a * b); // 30
```

```
printf("%d", a / b); // 3
```

```
printf("%d", a % b); // 1
```

Important Point

For integer division:

$$10 / 3 = 3$$

The decimal part is discarded.

6. Relational Operators

Relational operators are used to **compare two values**.

The result is either **true (1)** or **false (0)**.

Operator	Meaning	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal to	a >= b

Operator	Meaning	Example
<=	Less than or equal to	a <= b

Example

```
int a = 10, b = 20;
```

```
printf("%d", a < b);
```

Output:

1

Because $10 < 20$ is true.

Important Difference

```
a = 10;    // Assignment
```

```
a == 10;   // Comparison
```

= is an **assignment operator**, while == is a **relational operator**.

7. Logical Operators

Logical operators are used to combine or reverse conditions.

Operator	Name	Meaning
&&	Logical AND	Both conditions must be true
!	Logical NOT	Reverses the result

Logical AND &&

```
(a > 5 && b > 5)
```

True only when **both conditions are true**.

Logical OR ||

(a > 5 || b > 5)

True when **at least one condition is true**.

Logical NOT !

!(a > 5)

Reverses the result.

Truth Table

A	B	A && B	A B
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

NOT Truth Table

A	!A
0	1
1	0

Simple Expressions in C

An expression in C is a combination of constants, variables and operators that produces a value.

Example

a + b

$x * 5 + 10$

$(a + b) / 2$

1. Operator Precedence

Operator Precedence determines which operator is performed first when an expression contains multiple operators.

Example

`int result = 10 + 5 * 2;`

Here, `*` has higher precedence than `+`.

So:

$10 + 5 * 2$

$= 10 + 10$

$= 20$

Common Precedence Order

Priority	Operators	Description
1	()	Parentheses
2	* / %	Multiplication, Division, Modulus
3	+ -	Addition, Subtraction
4	< <= > >=	Relational
5	== !=	Equality
6	&&	Logical AND
7	?:	Conditional
8	=	Assignment

Higher priority number in the table means lower precedence.

Example

```
int x = 10 + 20 * 3;
```

First:

$$20 * 3 = 60$$

Then:

$$10 + 60 = 70$$

Therefore:

$$x = 70$$

2. Associativity

Associativity determines the direction in which operators of the same precedence are evaluated.

There are two directions:

- Left to Right
- Right to Left

Example: Left-to-Right

`int x = 20 / 5 * 2;`

/ and * have the same precedence.

Their associativity is left to right.

So:

$20 / 5 * 2$

$= 4 * 2$

$= 8$

Not:

$20 / 10 = 2$

Example: Right-to-Left

Assignment operators have right-to-left associativity.

`a = b = c = 10;`

Evaluation:

`c = 10`

`b = c`

`a = b`

Therefore:

`a = 10`

`b = 10`

`c = 10`

Precedence vs Associativity

Precedence	Associativity
Determines which operator is performed first	Determines direction of evaluation
Used when operators have different precedence	Important when operators have the same precedence
Example: * before +	Example: * and / are left-to-right

Precedence → Which operator first?

Associativity → Which direction?

Example

int x = 10 + 20 * 3 - 5;

Step 1: * first

10 + 60 - 5

Step 2: + and - have same precedence and are left-to-right

70 - 5

= 65

Answer: 65

When multiplication (*) and division (/) have the same precedence, C follows left-to-right associativity.

So, whichever appears first from the left is executed first.

Example 1

20 / 5 * 2

Both / and * have the same precedence.

From left to right:

$$20 / 5 * 2$$

$$= 4 * 2$$

$$= 8$$

Answer = 8

Example 2

$$20 * 5 / 2$$

From left to right:

$$20 * 5 / 2$$

$$= 100 / 2$$

$$= 50$$

Answer = 50

Operator Precedence and Associativity

Operator Precedence

Operator precedence determines **which operator is evaluated first** when an expression contains different operators.

For example:

$$5 + 2 * 3$$

* has higher precedence than +, so:

$$5 + (2 * 3)$$

$$= 5 + 6$$

$$= 11$$

Operator Associativity

Associativity determines the **order of evaluation** when operators have the same precedence.

For example:

$$20 / 5 * 2$$

/ and * have the same precedence and **left-to-right associativity**:

$$(20 / 5) * 2$$

$$= 4 * 2$$

$$= 8$$

Evaluate the Expression (Imp)

Given:

$$x = 5 * 2 - 8 / 4 + 3 > 6 \&\& 1$$

We apply the operators according to their precedence.

Step 1: Multiplication and Division

* and / have higher precedence than + and -.

$$5 * 2 = 10$$

$$8 / 4 = 2$$

Therefore:

$$x = 10 - 2 + 3 > 6 \&\& 1$$

Step 2: Addition and Subtraction

+ and - have the same precedence and are evaluated **left to right**.

$$10 - 2 + 3$$

$$= 8 + 3$$

$$= 11$$

Therefore:

$$x = 11 > 6 \&\& 1$$

Step 3: Relational Operator >

$$11 > 6$$

This is **true**, so in C:

$$11 > 6 = 1$$

Therefore:

$$x = 1 \ \&\& \ 1$$

Step 4: Logical AND &&

$$1 \ \&\& \ 1 = 1$$

Therefore:

Final Answer

$$x = 1$$

Complete Order of Evaluation

$$5 * 2 - 8 / 4 + 3 > 6 \ \&\& \ 1$$

↓

$$10 - 2 + 3 > 6 \ \&\& \ 1$$

↓

$$8 + 3 > 6 \ \&\& \ 1$$

↓

$$11 > 6 \ \&\& \ 1$$

↓

$$1 \ \&\& \ 1$$

↓

$$1$$

Final value: x = 1

TYPE

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int num1 = 2, num2 = 4, num3 = 7;
```

```
    float sum, avg;
```

```
    float num4= 7.5;
```

```
    sum = num1 + num2 + num3 + num4;
```

```
    avg = (float)sum / 3;
```

```
    printf("\nThe Sum of Three Numbers    = %f", sum);
```

```
    printf("\nThe Average of Three Numbers = %.2f\n", avg);
```

```
}
```

```
#include <stdio.h>
```

```
int main() {
```

```
    // Variable declarations
```

```
    float sub1 = 70, sub2= 80, sub3= 50, sub4= 78, sub5= 94;
```

```
    float total, percentage;
```

```
    // Calculate total and percentage
```

```
    total = sub1 + sub2 + sub3 + sub4 + sub5;
```

```
    percentage = (total / 500.0) * 100.0;
```

```
    // Display the results
```

```
printf("\nTotal Marks Obtained: %.2f / 500.00\n", total);

printf("Percentage: %.2f%%\n", percentage);

return 0;

}
```

IF ELSE

```
#include <stdio.h>

int main() {

    int number = 70;

    if (number >= 50) {

        printf("number is an greater than 50.");

    }

    else {

        printf("number is an less than 50.");

    }

    return 0;

}
```

```
#include <stdio.h>

int main() {

    int number = 70;


    if (number == 50) {

        printf("number is an equal to 50.");

    }

    else if (number >= 50)
```

```

    {

        printf("number is an greater than 50.");

    }

    else {

        printf("number is an less than 50.");

    }

    return 0;

}

#include <stdio.h>

int main() {

    int number = 70;

    if (number >= 50) {

        printf("%d is an greater than 50.",number);

    }

    else {

        printf("%d is an less than 50.",number);

    }

    return 0;

}

// Check whether an integer is odd or even

#include <stdio.h>

int main() {

    int number;

```

```
if (number%2 == 0) {  
    printf("%d is an even integer.",number);  
}  
else {  
    printf("%d is an odd integer.",number);  
}  
return 0;  
}  
printf("Enter an integer: ");  
scanf("%d", &number);
```